

ASSIGNMENT 7

QUESTION 1

lorem

SOLUTION

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

def uniform_noise(image, a=-50, b=50):
    noise = np.random.uniform(a, b, image.shape).astype(np.float32)
    noisy_image = cv2.add(image.astype(np.float32), noise)
    return (np.clip(noisy_image, 0, 255).astype(np.uint8), noise)

def gaussian_noise(image, mean=0, sigma=25):
    noise = np.random.normal(mean, sigma, image.shape).astype(np.float32)
    noisy_image = cv2.add(image.astype(np.float32), noise)
    return (np.clip(noisy_image, 0, 255).astype(np.uint8), noise)

def erlang_noise(image, shape=2, scale=25):
    noise = np.random.gamma(shape, scale, image.shape).astype(np.float32)
    noisy_image = cv2.add(image.astype(np.float32), noise)
    return (np.clip(noisy_image, 0, 255).astype(np.uint8), noise)

def exponential_noise(image, scale=25):
    noise = np.random.exponential(scale, image.shape).astype(np.float32)
    noisy_image = cv2.add(image.astype(np.float32), noise)
    return (np.clip(noisy_image, 0, 255).astype(np.uint8), noise)

def rayleigh_noise(image, scale=25):
    noise = np.random.rayleigh(scale, image.shape).astype(np.float32)
    noisy_image = cv2.add(image.astype(np.float32), noise)
    return (np.clip(noisy_image, 0, 255).astype(np.uint8), noise)

img = cv2.imread("./img1.jpg")

plt.figure(figsize=(20, 10))
plt.subplot(5, 4, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

noisy_img = uniform_noise(img)
plt.subplot(5, 4, 2)
plt.imshow(cv2.cvtColor(noisy_img[0], cv2.COLOR_BGR2RGB))
plt.title("Uniform Noise Inbuilt")
plt.axis("off")

gauss_noisy_img = gaussian_noise(img)
```

```

plt.subplot(5, 4, 3)
plt.imshow(cv2.cvtColor(gauss_noisy_img[0], cv2.COLOR_BGR2RGB))
plt.title("Gaussian Noise Inbuilt")
plt.axis("off")

erlang_noisy_img = erlang_noise(img)
plt.subplot(5, 4, 4)
plt.imshow(cv2.cvtColor(erlang_noisy_img[0], cv2.COLOR_BGR2RGB))
plt.title("Erlang Noise Inbuilt")
plt.axis("off")

exp_noisy_img = exponential_noise(img)
plt.subplot(5, 4, 5)
plt.imshow(cv2.cvtColor(exp_noisy_img[0], cv2.COLOR_BGR2RGB))
plt.title("Exponential Noise Inbuilt")
plt.axis("off")

rayleigh_noisy_img = rayleigh_noise(img)
plt.subplot(5, 4, 6)
plt.imshow(cv2.cvtColor(rayleigh_noisy_img[0], cv2.COLOR_BGR2RGB))
plt.title("Rayleigh Noise Inbuilt")
plt.axis("off")

plt.subplot(5, 4, 7)
plt.hist(noisy_img[1].ravel(), bins=256, color="blue", alpha=0.7)
plt.title("Uniform Noise Histogram")

plt.subplot(5, 4, 8)
plt.hist(gauss_noisy_img[1].ravel(), bins=256, color="green", alpha=0.7)
plt.title("Gaussian Noise Histogram")

plt.subplot(5, 4, 9)
plt.hist(erlang_noisy_img[1].ravel(), bins=256, color="red", alpha=0.7)
plt.title("Erlang Noise Histogram")

plt.subplot(5, 4, 10)
plt.hist(exp_noisy_img[1].ravel(), bins=256, color="purple", alpha=0.7)
plt.title("Exponential Noise Histogram")

plt.subplot(5, 4, 11)
plt.hist(rayleigh_noisy_img[1].ravel(), bins=256, color="orange", alpha=0.7)
plt.title("Rayleigh Noise Histogram")

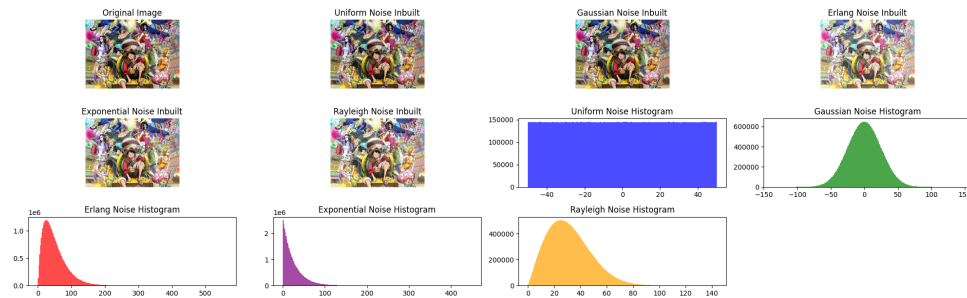
plt.tight_layout()

plt.show()

```

OUTPUT

<Figure size 2000x1000 with 11 Axes>



QUESTION 2

Write a program to use random values in confusion matrix and compute the different quantitative metrics (Accuracy, precision, recall, F1-score, MCC) for 2x2 and 3x3 matrices.

SOLUTION

```
import numpy as np

matrix_2x2 = np.random.randint(0, 100, size=(2, 2))
matrix_3x3 = np.random.randint(0, 100, size=(3, 3))

metrics = ["Accuracy", "Precision", "Recall", "F1-Score", "MCC"]

print(matrix_2x2)
print("For 2x2 Matrix")
TP = matrix_2x2[0][0]
TN = matrix_2x2[1][1]
FP = matrix_2x2[1][0]
FN = matrix_2x2[0][1]

accuracy = (TP + TN) / (TP + TN + FP + FN)
print("Accuracy", accuracy)

precision = TP / (TP + FP)
print("Precision", precision)

recall = TN / (TN + FN)
print("Recall", recall)

f1_score = (2 * precision * recall) / (precision + recall)
print("F1-Score", f1_score)

mcc = ((TP * TN) - (FP * FN)) / np.sqrt((TP + FP) * (TP + FN) * (TN + FP) * (TN + FN))
print("MCC", mcc)

print(matrix_3x3)
print("For 3x3 matrix")

tp = matrix_3x3[0][0]
fn = matrix_3x3[0][1] + matrix_3x3[0][2]
fp = matrix_3x3[1][0] + matrix_3x3[2][0]
tn = matrix_3x3[1][1] + matrix_3x3[1][2] + matrix_3x3[2][1] + matrix_3x3[2][2]

accuracy = (tp + tn) / (tp + tn + fp + fn)
print("Accuracy", accuracy)
```

```

precision = tp / (tp + fp)
print("Precision", precision)

recall = tp / (tp + fn)
print("Recall", recall)

f1_score = (2 * precision * recall) / (precision + recall)
print("F1-Score", f1_score)

mcc = ((tp * tn) - (fp * fn)) / np.sqrt((tp + fp) * (tp + fn) * (tn + fp) * (tn + fn))
print("MCC", mcc)

```

OUTPUT

```

[[51 13]
 [57 45]]
For 2x2 Matrix
Accuracy 0.6506024096385542
Precision 0.4722222222222222
Recall 0.796875
F1-Score 0.5930232558139534
MCC 0.24301611129116804
[[40 29 55]
 [22 79 73]
 [46 61 37]]
For 3x3 matrix
Accuracy 0.24434389140271492
Precision 0.37037037037037035
Recall 0.3225806451612903
F1-Score 0.3448275862068965
MCC 0.1136960201982203

```